Week 1: Can Machines Think?

ELIZA to ChatGPT, and what this unit teaches you to ask

FIT1061: Introduction to Artificial Intelligence

2026-07-13

The Story

In 1966, Joseph Weizenbaum, a computer scientist at MIT, wrote a program called **ELIZA**. It was about 200 lines of LISP. The default script, called DOCTOR, mimicked a Rogerian psychotherapist: if you typed "I am sad", it might reply "WHY ARE YOU SAD?", or "HOW OFTEN ARE YOU SAD?", or "I AM SORRY TO HEAR YOU ARE SAD." Each input pattern had a small list of response templates, and ELIZA picked one in turn, so the same sentence could produce different replies on different days. If you typed "My mother doesn't understand me", it would reply "TELL ME MORE ABOUT YOUR FAMILY".

ELIZA worked by pattern-matching. It had no model of conversation, no memory across turns, and no understanding of the words it produced. It just rearranged the user's input into a question. And yet Weizenbaum's secretary, who had watched him build the program, asked him to leave the room so she could "talk to it in private". She knew exactly how it worked, and it worked on her anyway.

Sixty years later, ChatGPT can hold a thirty-minute conversation that is (for most people, most of the time) indistinguishable from a thoughtful human correspondent. The machinery is eight orders of magnitude more elaborate (hundreds of billions of parameters, trained on trillions of tokens of text, at a cost of around $100 million). The illusion is far more convincing.

The question this unit opens with is: **does that difference matter?**

Can Machines Think?

The imitation game

In 1950, Alan Turing published a paper titled *Computing Machinery and Intelligence*. He opened with the question "Can machines think?" and then immediately refused to answer it.

The terms, he argued, were too vague: "thinking" means too many different things for the question to have a single answer.

Instead, he proposed to **replace** the question with a game. The game has three players: a human judge, a human respondent, and a machine respondent. The judge can communicate with both, but only through text; they cannot see, hear, or otherwise observe either. After a fixed amount of conversation (Turing suggested five minutes), the judge guesses which respondent is the machine.

The rule: **if the judge cannot reliably tell which is which, then for any practical purpose, the machine "thinks".**

This is Turing's move. He didn't claim to define thinking. He proposed a way to operationalise it: to turn an unanswerable metaphysical question into a testable behavioural one.

Fooling people

ELIZA fooled people in 1966. Not because it passed the Turing test in a rigorous sense: the conversations were too short, too narrowly scripted, the script was a therapist (which conveniently turns every input back into a question), and Weizenbaum's secretary already knew it was a machine. But the *psychological* effect was real: people projected intelligence onto a 200-line pattern matcher.

In 2014, a chatbot called **Eugene Goostman** was claimed to have "passed the Turing test" at a competition in London. Goostman posed as a 13-year-old non-native English speaker: a persona that lets a chatbot deflect questions about real-world knowledge ("I'm just a kid", "my English isn't great"). It fooled 33% of judges in five-minute chats. The headlines declared the Turing test passed.

The experts disagreed. They pointed out that the test format was too short, the persona was too narrow, and the judges were too easily fooled. *"That's not the test we meant."*

Would ChatGPT pass?

By 2022, GPT-3.5 could hold open, broad conversations that fooled most people most of the time. In 2023, GPT-4 was reported to score in the top 10% on the US bar exam, pass medical licensing tests, and score the maximum on multiple university-entry standardised tests. In 2024–25, "reasoning models" like OpenAI's o1 and DeepSeek's R1 began solving Olympic-level mathematics problems.

By Turing's original criterion (*can a machine engage in textual conversation such that a judge cannot reliably distinguish it from a human?*) the test really has been passed in the broad sense. Not by the trick of a narrow persona; by general fluency.

So the question Turing's framing was supposed to retire is still hanging.

Melanie Mitchell, an AI researcher and author of *Artificial Intelligence: A Guide for Thinking Humans*, calls "intelligence" a **suitcase word**. The suitcase, she says, is too small for the meanings packed inside it: pattern recognition, abstract reasoning, language, social judgement, embodied skill, planning, creativity, self-awareness. When we say "ChatGPT is intelligent" or "ChatGPT is not intelligent", we are saying something about *some* of those things and silently dropping others.

Turing's question (*can machines think?*) quietly became *what does* thinking *mean?*

Is It AI?

If the philosophical question is unanswerable as posed, what should we do?

This unit's answer: **stop asking it.** We will not use "intelligent" or "AI" as a definition for this unit. We will use **algorithms**.

To see why, consider these ten systems. For each, ask yourself: is this AI?

#	System	What it does
1	Mechanical thermostat	Bimetallic coil tilts a mercury switch. No code.
2	Nest learning thermostat	Adjusts the schedule from your behaviour. Updates internal parameters.
3	Google Maps routing	Finds the shortest path on a graph. Algorithm published 1959.
4	Word spellcheck	Dictionary lookup + edit distance. Was AI in the 1980s.
5	Spam filter	Estimates P(spam | words) from past examples.
6	Roomba	Maps the room. Plans a path. Doesn't learn.
7	Tesla Autopilot	Camera + neural network steering.
8	AlphaFold	Predicts 3D protein structures. Won the 2024 Nobel.
9	COMPAS recidivism score	137 inputs → risk score. Used in US courts.

#	System	What it does
10	ChatGPT writing these lecture notes	Generates the words you're reading.

You will find that the line between "AI" and "not AI" doesn't sit anywhere in particular. The Roomba *plans* but doesn't *learn*. Spellcheck was AI when it was new and isn't AI now, even though it does the same thing. AlphaFold solved a 50-year-old open problem in biology but most people don't think of it as AI because it isn't conversational. The cards do not separate cleanly into two piles.

John McCarthy, who coined the term *artificial intelligence* in 1956, defined it as:

> *AI is the science of making machines do things that would require intelligence if done by humans.*

But the same McCarthy later observed:

> *As soon as it works, no one calls it AI anymore.*

Seventy years later, Arvind Narayanan and Sayash Kapoor make the same observation:

> *AI is whatever hasn't been done yet.*

If the label keeps slipping away from working systems, the label is doing no work. What's left is the underlying machinery: the **algorithms**. That's what this unit teaches.

The Three Blocks

Every algorithm in this unit answers one of three questions:

i The three big questions

How do machines search? (Block A, Weeks 2–5) When the answer is hidden inside a space of possibilities, how do we find it without checking everything? BFS, DFS, greedy best-first, A*, hill climbing, minimax.

How do machines decide under uncertainty? (Block B, Weeks 6–9) When the right answer is probabilistic, how do we estimate it from data and act on the estimate? Probability, Bayes' rule, Naive Bayes, weighted sums, fairness.

How do machines learn? (Block C, Weeks 10–12) When the right rule is unknown, how do we extract it from many examples? Gradient descent, perceptron training, convolution, deployment.

One flagship system per block illustrates the recipe:

- **Block A: AlphaGo (DeepMind, 2016)** beat Lee Sedol 4–1 at Go. The recipe was *search + learning*: a neural network proposes promising moves, a tree search (MCTS) explores their consequences, self-play feeds the network better data. The same recipe drives AlphaProof (IMO silver, 2024), Gemini Deep Think (IMO gold, 2025), and every modern agent that plans (Cursor, Claude Code, ChatGPT Agents).
- **Block B: Watson (IBM, 2011)** beat the Jeopardy! champions Brad Rutter and Ken Jennings. The recipe was *probability over evidence*: estimate $P(\text{outcome} \mid \text{data})$ from past examples, combine features into a decision, then audit who the model gets right and wrong. The same machinery (with very different stakes) runs COMPAS in US courts. (We meet that case in Week 9.)
- **Block C: AlphaFold (DeepMind, 2024 Nobel Prize)** predicts the 3D shape of any protein. The recipe was *gradient descent at scale*: start with millions of labelled examples, define a loss, nudge the weights downhill billions of times. The same recipe behind ChatGPT, Tesla Autopilot, GraphCast (weather), and AlphaFold itself.

And LLMs?

Modern Large Language Models (ChatGPT, Claude, Gemini) combine **all three blocks**.

- **Block A (search):** when an LLM is wrapped in an *agent* (Cursor, Claude Code, ChatGPT Agents), it searches over possible actions. Reasoning models like o1 and R1 also search, over chains of thought.
- **Block B (probability):** next-token prediction is, literally, a conditional probability. The perceptron (a weighted sum) is the transformer's unit cell.
- **Block C (learning):** gradient descent trained the weights. Further training on human feedback (RLHF) polished the behaviour.

You will not build ChatGPT in this unit. You will build the **building blocks** that make it work, and the tools to ask whether it should be built.

How This Unit Works

Each week has a **milestone** (a historical AI event), a **Pass task** (P1 through P10), and optional **Credit, Distinction, and HD extensions**. You pick your tier. You can move up at any point before Week 13.

Tier	Grade band	Tasks
Pass	50–59	P1–P10. Implement each algorithm with scaffolding. Engage with the milestone.
Credit	60–69	Pass + 4 Credit extensions. Less scaffolding. More ambitious problems.
Distinction	70–79	Credit + 4 Distinction tasks. Includes A*, perceptron training, an adversarial game tournament.
HD	80+	Distinction + 1 comprehensive project. Problem definition → implementation → interview.

Two outcomes per task:

- **Complete**: task accepted. Locked in.
- **Fix and Resubmit**: feedback given. Try again. Every task allows as many attempts as you need until Week 13, used in good faith; your tutor may pause resubmissions that stop engaging with the feedback.

One hurdle test in Week 10 covers Pass tasks 2–9 (no W1, no devices, no code; short answer and hand-computation). A resit is offered in Week 12, and a supplementary in Week 14 (capped at Pass).

All AI tool use must be **cited**: tool name, prompt, date. The rules on what you may use a tool for scale by tier (explanation OK at Pass; debugging your own code OK at Credit; pair-programming OK at Distinction; HD requires *critical evaluation*). The full table is in the Assessment FAQ (on Moodle).

This Week's Lab

P1 has three parts and is graded on **honest engagement**, not correctness:

1. **P1.1 Setup.** Install Python, Jupyter, and an editor (VS Code, PyCharm, or your tool of choice). Verify the environment by running the W01 notebook end-to-end.
2. **P1.2 Learning Plan.** A short (~300 word) statement: which tier are you aiming for, and why? Honest reasoning matters more than the choice; you can revise it any time before Week 13.
3. **P1.3 Python Bridge.** A zero-stakes warm-up bridging the C++ you learned in FIT1045 to the Python you'll use all semester: one translation exercise plus a short bootcamp. Ungraded but required before P2.

You submit all three notebooks by end-of-week.

Three Questions, Every Week

This unit's refrain. Three questions you should ask of every algorithm we build, and every system you encounter outside the unit:

> The unit's three questions
>
> 1. **How does it work?**
> 2. **When does it fail?**
> 3. **Who does it affect?**

See you in Week 2.